



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO



Patricio Ignacio Contreras Bruna

Analista de conjunto de datos e implementación de VLM en Proyecto Fondecyt

Informe de Práctica Profesional de Ingeniero Civil Electrónico

Fecha de práctica: 03/06/2024 - 03/08/2024



**Escuela de Ingeniería Eléctrica
Facultad de Ingeniería**

Valparaíso, 29 de Agosto del 2024

Resumen

La práctica profesional se llevó a cabo en el Laboratorio de Robótica y Visión por Computadora, situado en Avenida Brasil 2147, en la Escuela de Ingeniería Eléctrica. Esta tuvo una duración de dos meses, comenzando el 3 de junio y finalizando el 3 de agosto de 2024. La supervisión estuvo a cargo del profesor Gabriel Hermosilla, con el objetivo principal de analizar datos satelitales de depósitos mineros en Chile e implementar un modelo de inteligencia artificial mediante fine-tuning para el análisis automático de dichos datos.

Para lograr estos objetivos, se utilizó el servidor de la Escuela de Ingeniería Eléctrica, que cuenta con dos tarjetas de video H100, esenciales para el entrenamiento, la inferencia y el fine-tuning de modelos de inteligencia artificial como los LLM (Large Language Models) y VLM (Visual Model Language).

Esta práctica se desarrolló en el marco del proyecto Fondecyt N°1240573 - AI-Minesafe: Safety in Mining Waste Management Through Artificial Intelligence, que se enfoca en el análisis de desperdicios mineros y la implementación del Visual Model Language Florence-2.

Objetivo general

- Realizar análisis de datos satelitales de depósitos mineros e integrar modelo de Inteligencia Artificial.

Objetivos específicos

- Analizar los datos satélites de las mineras de Chile.
- Aprender como opera el Visual Model Language Florence-2.
- Integrar el VLM para el análisis automático de depósitos mineros vía satelital.

Palabras clave: Depositos mineros, datos satelitales, inteligencia artificial, LLM, VLM.

Índice general

1	Información sobre la empresa	1
1.1	Generalidades de la Empresa	1
1.2	Organización General de la Empresa	1
1.2.1	Organigrama del Equipo Directivo	1
1.2.2	Procesos Principales	2
1.2.3	Control de Calidad	3
1.3	Laboratorio de Robótica y Visión por Computadora	3
2	Trabajo Realizado	5
2.1	Carta Gantt	5
2.2	Trabajo realizado	5
2.2.1	Actividad 1 - Análisis de imágenes satelitales	5
2.2.2	Actividad 2 - Florence-2	8
2.2.3	Modelo Florence-2	8
2.2.4	Actividad 3 - Implementación de Florence-2 en proyecto Fondecyt	10
	Discusión y conclusiones	16
	Bibliografía	19
A	Apéndice	A-1
A.1	Importar librerías	A-1
A.2	Cargar Modelo	A-2
A.3	Cargar dirección de Conjunto de Datos	A-2
A.4	Configurar LoRa del Modelo	A-3
A.5	Realizar inferencias entre épocas	A-4
A.6	Definir Loop de Entrenamiento Fine-tuning	A-5
A.7	Ejecutar entrenamiento	A-6
A.8	Recolectar predicciones para métricas	A-6
A.9	Calcular Mean Average Precision	A-7
A.10	Guardar Modelo Fine-tuned	A-7

1 Información sobre la empresa

La Escuela de Ingeniería Eléctrica de la Pontificia Universidad Católica de Valparaíso imparte cinco carreras de pregrado, sumado a lo anterior donde se imparte los programa de postgrado.

1.1 Generalidades de la Empresa

A continuación, se presenta en resumen las generalidades del departamento donde se realizó la práctica profesional.

- a) Escuela de Ingeniería Eléctrica (EIE), específicamente en el Laboratorio de Robótica
- b) AV. Brasil 2147, 2362804 Valparaíso
- c) Números de empleados ver Tabla 1.1
- d) Gabriel Hermosilla, Dr. Ingeniería Eléctrica, Profesor Adjunto, gabriel.hermosilla@pucv.cl

Tabla 1.1: Empleados de la Escuela de Ingeniería Eléctrica.

Equipo	Total de empleados
Profesores Jornada Completa	22
Personal de Jornada Parcial	19
Personal Administrativo	10
Personal de Servicio	3

1.2 Organización General de la Empresa

1.2.1 Organigrama del Equipo Directivo

En el organigrama de la Figura 1.1 se presenta en detalle la composición del **equipo directivo** de la Escuela de Ingeniería Eléctrica, los cuales además pertenecen al grupo de profesores de jornada completa.

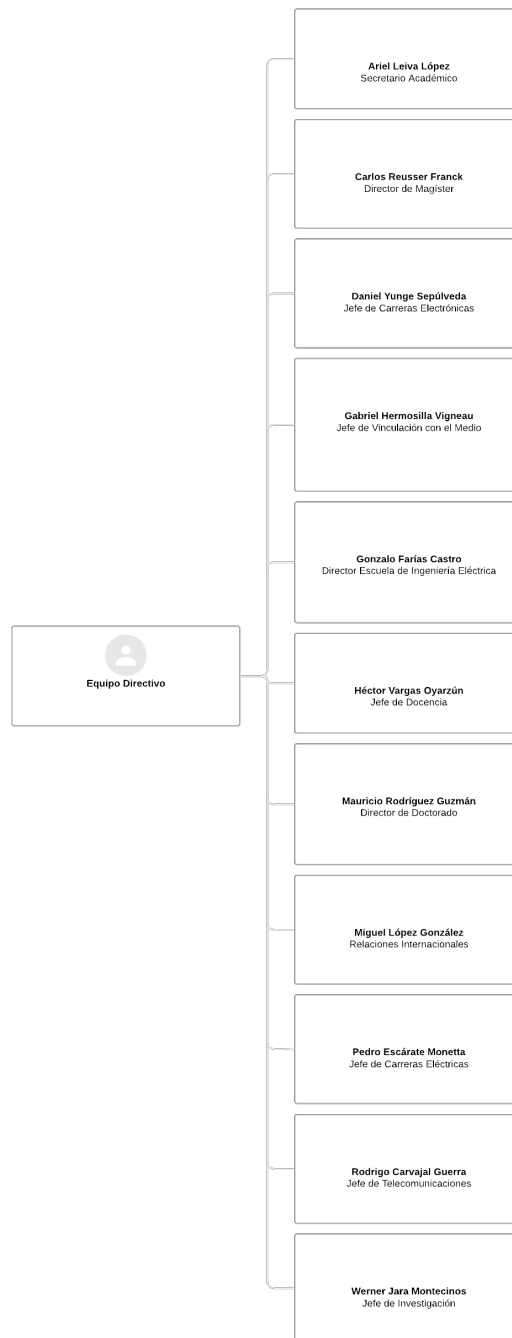


Figura 1.1: Organigrama del Equipo Directivo de la Escuela de Ingeniería Eléctrica.

1.2.2 Procesos Principales

Los procesos principales de la Escuela es la educación, formar profesionales de las cinco carreras que componen la escuela, tales como:

- Ingeniería Civil Electrónica
- Ingeniería Civil Eléctrica
- Ingeniería Civil en Telecomunicaciones
- Ingeniería Electrónica
- Ingeniería Eléctrica

Formar profesionales de postgrado, tales como:

- Magíster en Ciencias de la Ingeniería con mención en Ingeniería Eléctrica
- Doctorado en Ingeniería Eléctrica

1.2.3 Control de Calidad

En la Escuela de Ingeniería Eléctrica se encuentra el Laboratorio de Control de Calidad y Fotometría, cuyo objetivo es certificar productos eléctricos relacionados con la iluminación. Esto incluye la evaluación y certificación de luminarias y sistemas de alumbrado público utilizados en las calles.

El Laboratorio de Fotometría y Control de Calidad participa en dos proyectos de la Agencia Nacional de Investigación y Desarrollo (ANID), en colaboración con el Laboratorio LabSens de la misma escuela. Estos proyectos cuentan con la participación del Ministerio de Medio Ambiente y la Superintendencia del Medio Ambiente.

El equipo que integra el Laboratorio de Control de Calidad y Fotometría se detalla en la Tabla 1.2.

Tabla 1.2: Equipo del Laboratorio de Control de Calidad y Fotometría.

Cargo	Nombre	Correo	Número
Jefe de laboratorio	Iván Kopaitic	ivan.kopaitic@pucv.cl	+56 (32) 2273756
Procesamiento de datos	Carla Tureo	carla.tureo@pucv.cl	+56 (32) 2273784
Seguridad Eléctrica	Jonathan Soto	jonathan.soto@pucv.cl	+56 (32) 2273735
Ensayos fotométricos	Andy Erices	andy.erices@pucv.cl	+56 (32) 2273780
Contabilidad	Cristián Araya	cristian.araya@pucv.cl	+56 (32) 2273689

1.3 Laboratorio de Robótica y Visión por Computadora

El Laboratorio de Robótica, dirigido por el profesor Gabriel Hermosilla, se enfoca en actividades relacionadas con la Robótica y la Inteligencia Artificial. Este laboratorio ofrece a los estudiantes de pregrado la oportunidad de realizar proyectos de tesis y participar activamente en diversas actividades de investigación. Además, proporciona a los estudiantes la posibilidad de continuar sus estudios en los programas de Magíster y Doctorado.

1 Información sobre la empresa

Durante el año 2024, el equipo del laboratorio está compuesto por cinco estudiantes de posgrado, seis estudiantes de pregrado en fase de tesis y cuatro estudiantes de pregrado. En la Tabla 1.3 detalla los equipos esenciales para el entrenamiento de modelos de inteligencia artificial y los dispositivos electrónicos utilizados en el desarrollo de la robótica. La Figura 1.2 ilustra la entrada al laboratorio junto con el robot de servicio

Tabla 1.3: Equipos electrónicos del Laboratorio de Robótica.

Equipos	Características
1 x Tarjeta Gráfica GeForce RTX 4090	24 GB de GPU
2 x Tarjeta Gráfica GeForce RTX 4070 Ti	16 GB de GPU
2 x Tarjeta Gráfica GeForce RTX 3060	12 GB de GPU
1 X Tarjeta Gráfica GeForce RTX 1650	8 GB de GPU
1 X Tarjeta Gráfica GeForce RTX 1050	4 GB de GPU
Tarjeta Arduino	-
Khepera Mobile Robot	-
Nvidia Jetson Nano Developer Kit	4 GB de GPU
ESP32	Wi-Fi/Bluetooth
Cámaras Térmicas	-
Multímetros	-
Osciloscopios	-
Robot de Servicio	-
Impresoras 3D	-
Smart TV Soporte	-



Figura 1.2: Laboratorio de Robótica y Visión.

2 Trabajo Realizado

A continuación, se detallan las actividades llevadas a cabo a lo largo de los dos meses de práctica. Se presenta la carta Gantt, seguida de una descripción detallada de cada actividad realizada.

2.1 Carta Gantt

En la Figura 2.1 se presentan todas las actividades realizadas durante la práctica profesional, que incluyeron el análisis de imágenes satelitales enfocadas en botaderos mineros y la posterior implementación del Visual Model Language (VLM) Florence-2.

		Junio				Julio			
		Semana 1	Semana 2	Semana 3	Semana 4	Semana 5	Semana 6	Semana 7	Semana 8
Actividad 1	Recolección de datos de imágenes satelitales								
	Preprocesamiento de las imágenes								
	Etiquetado manual de bboxes y segmentación de áreas en los botaderos mineros								
Actividad 2	Estudio de la arquitectura del modelo VLM								
	Integración con los datos manuales etiquetados								
Actividad 3	Realizar el fine-tuning del VLM usando los datos etiquetados manualmente								
	Evaluación y ajuste del modelo								
	Generación de resultados y análisis de desempeño								
	Validación del modelo fine-tuneado en nuevas imágenes satelitales								
	Generación de informes de desempeño								

Figura 2.1: Diagrama Gantt de las actividades realizadas.

2.2 Trabajo realizado

Los códigos correspondientes a la implementación de los sistemas se encuentran en el apartado 'Anexo'.

2.2.1 Actividad 1 - Análisis de imágenes satelitales

La práctica se desarrolló en el marco del Proyecto Fondecyt, enfocado en la localización de botaderos mineros dentro de diversas localidades de Chile. La primera actividad consistió en el análisis de imágenes satelitales para identificar correctamente los distintos tipos de residuos mineros, los cuales se clasificaron en botaderos estériles mineros, rípios de lixiviación y tranques de relave. En la Figura 2.2 se muestra la diferencia entre estos tipos de depósitos.



Figura 2.2: Ejemplo visual de botaderos mineros esteriles, ripio de lixiviacion y tranque de relave en minería

Para comenzar, se descargó un conjunto de imágenes satelitales de todos los depósitos mineros a lo largo de Chile, cada una con una resolución de 2400x2400 píxeles. Este conjunto de datos requirió aproximadamente 100 GB de espacio en el disco duro para su almacenamiento y análisis en un equipo determinado.

Con el fin de realizar un análisis adecuado y mantener un registro ordenado, se utilizó el software LabelStudio. Este programa permitió obtener tanto los cuadros delimitadores (bboxes) de cada depósito como las máscaras de segmentación. La interfaz del software se muestra en la Figura 2.3.

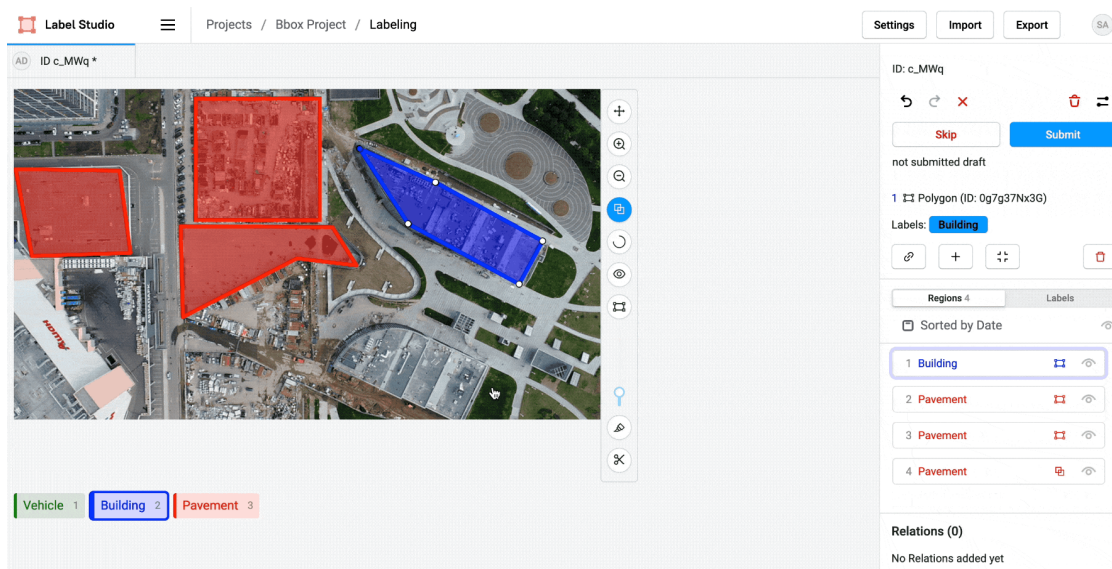


Figura 2.3: Interfaz label studio con imagen de prueba

Mediante el software mencionado, se analizaron aproximadamente 14,546 imágenes satelitales que contenían depósitos mineros. Un ejemplo de una vista satelital con los depósitos mencionados se muestra en la Figura 2.4, mientras que en la Figura 2.5 se presentan los depósitos ya analizados y segmentados..



Figura 2.4: Ejemplos de imágenes satelitales de los depósitos mineros



Figura 2.5: Ejemplos de imágenes satelitales de los depósitos mineros segmentados

2.2.2 Actividad 2 - Florence-2

Dado que la representación universal para diversas tareas relacionadas con la visión presenta desafíos únicos, principalmente en la necesidad de habilidades de comprensión perceptual, la visión por computadora se distingue del procesamiento de lenguaje natural (NLP, por sus siglas en inglés), que se centra en el manejo de texto[1]. La visión por computadora, por el contrario, requiere el procesamiento de datos visuales, como la localización de objetos, contornos de máscaras y características específicas. La representación de la visión por computadora se organiza en dos dimensiones clave:

- **Jerarquía Espacial:** El modelo debe ser capaz de distinguir los detalles espaciales en diferentes escalas, comprendiendo tanto los niveles conceptuales de las imágenes como los píxeles específicos de alta granularidad. Para capturar adecuadamente la jerarquía espacial, el modelo debe manejar múltiples niveles de granularidad en los datos visuales.
- **Granularidad Semántica:** La representación universal en visión por computadora debe abarcar un espectro de granularidad semántica, transitando desde descripciones generales hasta detalles más finos, lo que permite una comprensión versátil en diversas aplicaciones.

El gran desafío residía en la falta de un framework preentrenado unificado, capaz de integrar perfectamente la jerarquía espacial y la granularidad semántica en una única red para visión por computadora. Los modelos tradicionales se destacan en tareas individuales como la detección de objetos[2], la segmentación semántica[3] o la descripción de imágenes[4], pero era necesario desarrollar un modelo que pudiera abordar todas estas tareas en conjunto dentro de un mismo framework.

En respuesta a esta necesidad, se creó el modelo Florence[5], diseñado como un sistema unificado capaz de realizar todas estas tareas dentro de un único modelo. A diferencia de los enfoques anteriores, Florence integra aspectos espaciales, temporales y multimodales en un solo modelo de visión por computadora.

2.2.3 Modelo Florence-2

Como se mencionó anteriormente, el Modelo Florence-2[6] es capaz de abordar una amplia variedad de tareas de visión utilizando una arquitectura unificada[7] y un conjunto único de pesos. Este modelo fue preentrenado en un dataset llamado FLD-5B, que contiene 5.4 mil millones de anotaciones de comprensión y 124 millones de imágenes. Todas las anotaciones fueron estandarizadas uniformemente para generar salidas en formato de texto, facilitando el aprendizaje multitarea unificado mediante una función de optimización objetiva, descrita por la función de pérdida en la Ecuación (2.1).

$$\mathcal{L} = - \sum_{i=1}^{|y|} \log P_{\theta}(y_i | y < i, x), \quad (2.1)$$

El modelo opera de la siguiente manera: recibe una imagen de entrada junto con un "prompt" que define la tarea a realizar[8], y genera los resultados esperados en formato de texto. Es decir, la activación de tareas es obtenida mediante prompts de texto, reflejando la aproximación de los Large Language Models[9]. Luego, Utiliza un encoder de visión para convertir las imágenes en embeddings de tokens visuales, que posteriormente se concatenan con embeddings de texto y son procesados por un modelo encoder-decoder basado en transformers multimodales para generar la respuesta. El esquema del modelo se muestra en la Figura 2.6

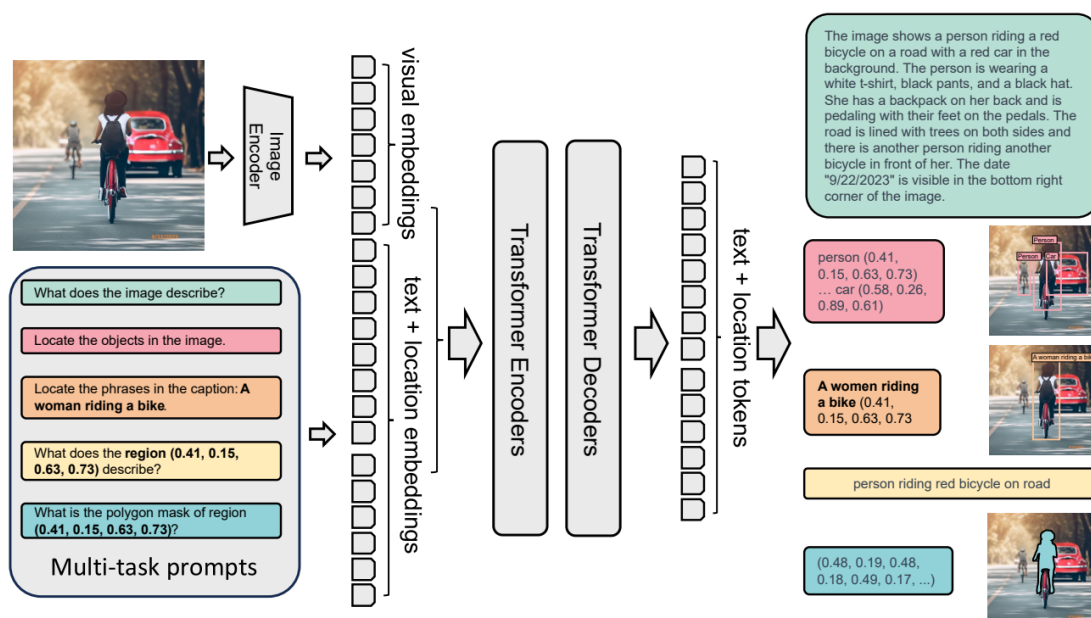


Figura 2.6: Esquema de Funcionamiento Modelo Florence 2

Florence-2 ofrece dos tipos de salida para las diversas tareas que realiza:

- **Texto:** Cuando el prompt de entrada es un texto plano sin formato especial, el modelo entrega una salida secuencial.
- **Región:** Para tareas específicas basadas en regiones, existen tokens de localización dentro del vocabulario del tokenizador. Estas localizaciones están representadas como coordenadas cuantizadas en 1,000 bins[10].

El modelo es capaz de realizar un total de 12 tareas, que pueden dividirse en aquellas con salida en texto y aquellas con salida en regiones. A continuación, se detallan las tareas con salida en texto[11], en las cuales la entrada es una imagen:

- **Descripción:** Proporciona una descripción general de lo que se observa en la imagen.
- **Descripción detallada:** Ofrece una descripción más precisa de los objetos presentes en la imagen.
- **Descripción más detallada:** Describe detalladamente los objetos presentes, su ubica-

ción y el paisaje.

- **Descripción de región:** Además de la imagen de entrada, se asigna una región específica, y el modelo genera una descripción de lo que se encuentra en esa región.

Para las tareas con salidas en regiones, el modelo realiza las siguientes:

- **Propuesta de Región:** El modelo propone una región de interés en la imagen y la detecta mediante un cuadro delimitador (bounding box).
- **Detección de Objetos:** Detecta los objetos presentes en la imagen y entrega las regiones correspondientes donde estos se encuentran.
- **Descripción densa de Región:** Similar a la detección de objetos, pero además asocia características a cada objeto detectado.
- **Aterrizaje a una frase:** El modelo localiza objetos en la imagen en función de una descripción textual proporcionada por el usuario[12].
- **Comprensión de expresión de referencia:** A partir de una entrada de texto, el modelo detecta el objeto que coincide con las características proporcionadas.
- **Detección de vocabulario abierto:** Dado un texto que describe un objeto, persona o personaje presente en la imagen, el modelo lo identifica y detecta.
- **Segmentación Semántica:** Mediante una entrada de texto que describe un objeto, el modelo segmenta semánticamente la imagen para identificar y marcar las regiones correspondientes.
- **Reconocimiento y detección de texto:** El modelo es capaz de detectar y reconocer texto presente en la imagen.

2.2.4 Actividad 3 - Implementación de Florence-2 en proyecto Fondecyt

Como última actividad de la práctica, se utilizó, reentrenó y se hizo un ajuste fino (fine-tuning) del modelo Florence-2, partiendo inicialmente de un conjunto de datos de prueba para verificar su correcto funcionamiento. Finalmente, se empleó el conjunto de datos correspondiente al proyecto minero Fondecyt MineSafe para la identificación precisa de los botaderos mineros, junto con otras tareas ofrecidas por el modelo.

El proceso comenzó con la conexión al servidor de la Escuela de Ingeniería Eléctrica, donde se cargó el modelo base Florence-2 de Microsoft, que cuenta con 0.23 billones de parámetros. Se realizó una prueba inicial para observar el desempeño del modelo base en la detección de botaderos mineros. Tras ejecutar tareas de prueba en algunas de las imágenes previamente analizadas, se observó que el modelo tendía a detectar con mayor precisión las figuras geométricas más prominentes, pero generalizaba en las descripciones. En la mayoría de los casos, la descripción generada fue, "Esta es la vista satelital de un campo desierto". Las detecciones realizadas por el modelo base se muestran en la Figura 2.7.

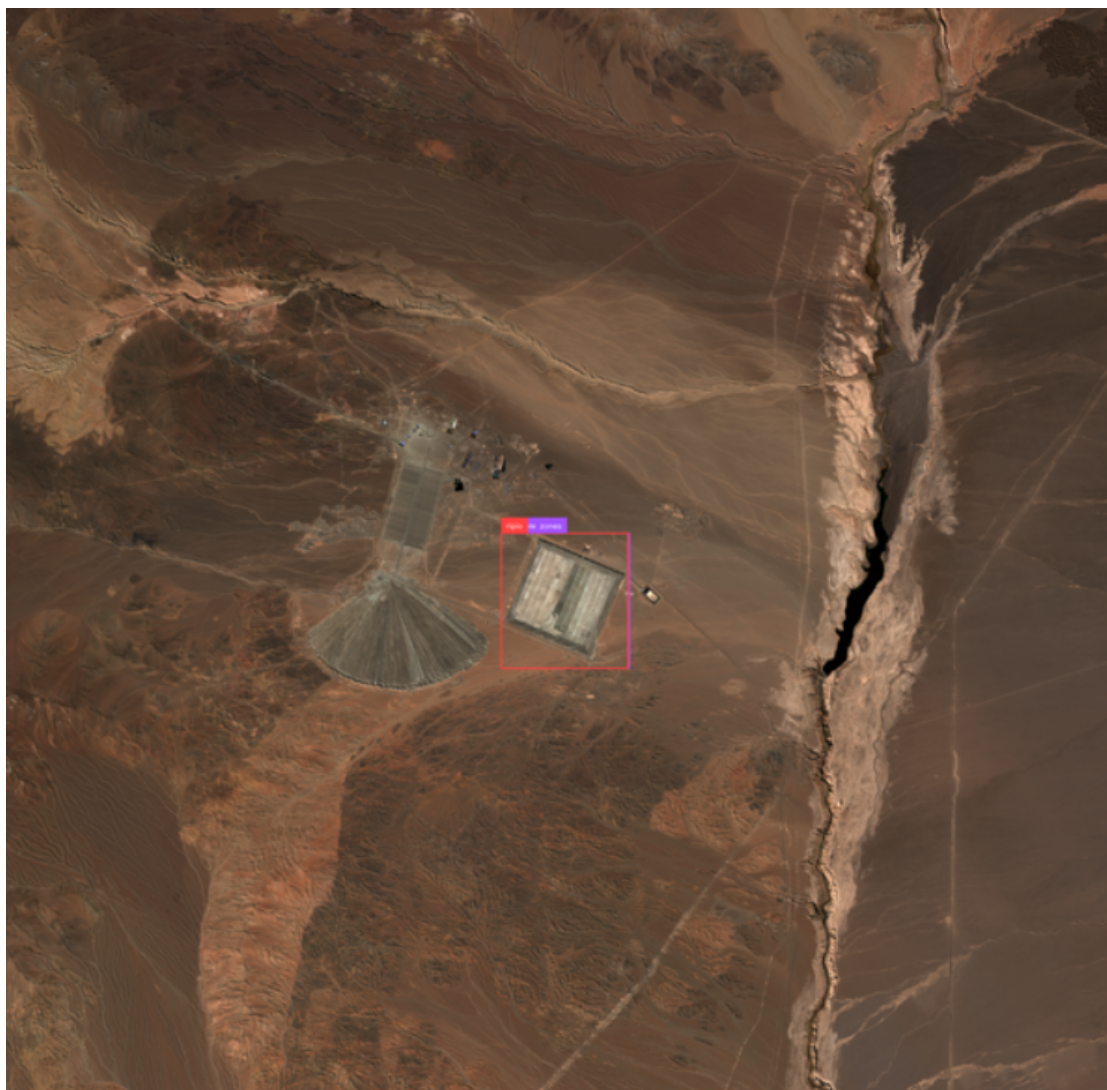


Figura 2.7: Prueba del Modelo Florence-2 base con imágenes de botaderos mineros desde un satélite

Con base en el modelo inicial proporcionado por Microsoft, no fue posible obtener un análisis automático preciso de los datos. Por lo tanto, fue necesario realizar un ajuste fino del modelo de visión y lenguaje para mejorar las detecciones y obtener descripciones más útiles. Esto se debe a que, al reentrenar una de las capas del modelo, las otras también se ven modificadas, lo que significa que al mejorar el rendimiento en una tarea, también se afecta el rendimiento en otras.

Para llevar a cabo el fine-tuning, se centró principalmente en la tarea de detección de objetos, ya que esta influye significativamente en las demás tareas del modelo. Se configuró un método LoRa (Low-Rank Adaptation) para el reentrenamiento, cuya función es congelar los datos ya entrenados en el modelo base y reentrenar los nuevos sin un consumo excesivo de GPU.

Aunque se disponía del servidor de la escuela, la GPU disponible no se compara con las que utilizan grandes empresas como Microsoft para desarrollar modelos de visión y lenguaje desde cero, por lo que el uso de LoRa fue crucial. De los 272,733,896 parámetros del modelo, solo el 70 % se utilizó durante el entrenamiento.

Se definió un proceso de entrenamiento utilizando el optimizador AdamW durante 10 épocas, con una tasa de aprendizaje de 0.000001. Durante cada época, se mostraron las imágenes para supervisar el progreso del modelo, monitoreando tanto la pérdida en el conjunto de entrenamiento como en el de validación.

Después del ajuste fino, se volvió a ejecutar Florence-2 utilizando la misma tarea de detección de objetos para evaluar su rendimiento. La Figura 2.8 muestra que el modelo logró un buen desempeño al detectar los depósitos mineros en una imagen aleatoria del conjunto de prueba. Este buen resultado puede atribuirse al ajuste específico realizado con el conjunto de datos, el cual incluía no solo imágenes de los depósitos, sino también su evolución temporal. Es decir, se contaba con múltiples imágenes del mismo depósito a lo largo del tiempo, permitiendo observar su crecimiento.



Figura 2.8: Tarea de Detección de Objetos en Depósitos mineros

Posteriormente, se procedió a probar tareas contiguas a la detección de objetos, como la segmentación semántica, cuyos resultados se observan en la Figura 2.9. Además, en la tarea de "Aterrizaje a frase", el modelo también fue capaz de detectar algunos depósitos mineros, aunque no todos, como se muestra en la Figura 2.10.

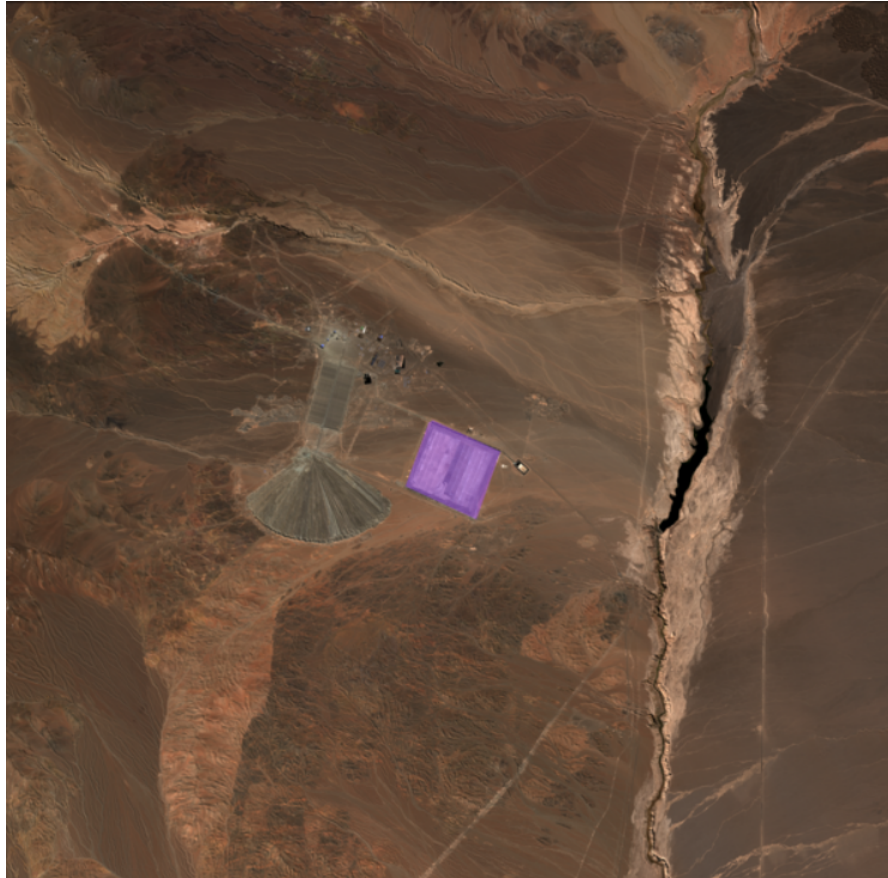


Figura 2.9: Tarea de Segmentación Semántica en Depósitos mineros



Figura 2.10: Tarea de Aterrizaje en frase en Depósitos mineros

Sin embargo, en otras tareas incluidas en Florence-2, el modelo no fue capaz de detectar correctamente los depósitos mineros ni proporcionar descripciones útiles. Por ejemplo, en muchas ocasiones, el resultado era simplemente "Vista aérea satelital de un desierto con un poblado y bem", donde "bem" hacía referencia a un botadero estéril minero, pero sin aportar información nueva o relevante a la descripción.

Hubiera sido útil realizar un ajuste fino adicional en otras tareas del VLM para mejorar la precisión de las respuestas del modelo en determinadas tareas. El principal obstáculo para realizar este tipo de entrenamientos fue la alta demanda del servidor. Aunque el servidor es capaz de realizar el fine-tuning, su uso es compartido con otros proyectos de estudiantes de posgrado, pregrado y profesores, lo que a menudo genera una cola de espera y limita el acceso durante periodos de alta ocupación. A pesar de estos inconvenientes, fue posible entrenar el modelo en las tareas correspondientes como la detección de objetos, logrando un

"Mean Average Precision" del 99 %, resultado del sobreajuste generado por el uso de imágenes temporales del mismo depósito.

Discusión y conclusiones

Durante el desarrollo de esta práctica profesional se implementaron técnicas avanzadas de visión por computadora y VLM, como Florence-2, para el análisis de imágenes satelitales enfocadas en la identificación y clasificación de botaderos mineros en Chile. El proceso incluyó la descarga y almacenamiento de un extenso conjunto de imágenes satelitales, la anotación de estas mediante el software LabelStudio, y el ajuste fino del modelo Florence-2 para obtener mejores resultados en la detección de objetos específicos como botaderos estériles, rípios de lixiviación y tranques de relave.

Inicialmente, se observó que el modelo preentrenado de Microsoft presentaba limitaciones para describir con precisión los botaderos mineros, emitiendo descripciones genéricas y detectando únicamente formas geométricas. Esto llevó a la necesidad de realizar un fine-tuning del modelo, enfocado principalmente en la tarea de detección de objetos. Mediante el uso de LoRa, se optimizó el proceso de ajuste fino para adaptarse a las limitaciones de hardware del servidor disponible, logrando una mejora notable en la precisión de las detecciones.

El ajuste fino permitió al modelo detectar depósitos mineros de manera más precisa, particularmente en la tarea de detección de objetos, mientras que otras tareas, como la segmentación semántica y el aterrizaje a frase, también mostraron resultados prometedores. Sin embargo, algunas tareas del modelo, como la descripción detallada y otras relacionadas, no lograron mejorar de manera significativa debido a las limitaciones del entrenamiento y la alta demanda del servidor, que dificultó la realización de entrenamientos más extensivos.

A pesar de estas limitaciones, los resultados obtenidos son satisfactorios, alcanzando un mAP del 99 %, principalmente debido al sobreajuste de los datos de entrenamiento que incluían imágenes temporales de los depósitos. Este enfoque temporal permitió observar la evolución de los botaderos a lo largo del tiempo, enriqueciendo el análisis del proyecto minero Fondecyt MineSafe.

En conclusión, la práctica profesional no solo permitió la implementación y ajuste fino de un modelo de última generación como Florence-2, sino que también evidenció los retos inherentes al uso de modelos preentrenados en tareas especializadas. Se destaca la necesidad de realizar ajustes adicionales en otras tareas del modelo para mejorar la calidad de las descripciones y segmentaciones, lo cual sería factible con mayor acceso a recursos computacionales. La experiencia adquirida en el manejo de modelos de visión y lenguaje contribuye al desarrollo

de soluciones más precisas y eficaces para el análisis de imágenes satelitales en el ámbito minero.

Bibliografía

- [1] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, y P. J. Liu, “Exploring the limits of transfer learning with a unified text-to-text transformer,” 2023. [En línea]. Disponible en: <https://arxiv.org/abs/1910.10683>
- [2] “Proceedings of ieee international conference on computer vision,” en *Proceedings of IEEE International Conference on Computer Vision*, 1995, pp. 0₁ – –.
- [3] B. Cheng, I. Misra, A. G. Schwing, A. Kirillov, y R. Girdhar, “Masked-attention mask transformer for universal image segmentation,” 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2112.01527>
- [4] J. Wang, Z. Yang, X. Hu, L. Li, K. Lin, Z. Gan, Z. Liu, C. Liu, y L. Wang, “Git: A generative image-to-text transformer for vision and language,” 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2205.14100>
- [5] L. Yuan, D. Chen, Y.-L. Chen, N. Codella, X. Dai, J. Gao, H. Hu, X. Huang, B. Li, C. Li, C. Liu, M. Liu, Z. Liu, Y. Lu, Y. Shi, L. Wang, J. Wang, B. Xiao, Z. Xiao, J. Yang, M. Zeng, L. Zhou, y P. Zhang, “Florence: A new foundation model for computer vision,” 2021. [En línea]. Disponible en: <https://arxiv.org/abs/2111.11432>
- [6] B. Xiao, H. Wu, W. Xu, X. Dai, H. Hu, Y. Lu, M. Zeng, C. Liu, y L. Yuan, “Florence-2: Advancing a unified representation for a variety of vision tasks,” 2023. [En línea]. Disponible en: <https://arxiv.org/abs/2311.06242>
- [7] P. Wang, A. Yang, R. Men, J. Lin, S. Bai, Z. Li, J. Ma, C. Zhou, J. Zhou, y H. Yang, “Unifying architectures, tasks, and modalities through a simple sequence-to-sequence learning framework,” *CoRR*, vol. abs/2202.03052, 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2202.03052>
- [8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, y I. Polosukhin, “Attention is all you need,” 2023. [En línea]. Disponible en: <https://arxiv.org/abs/1706.03762>
- [9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, y I. Sutskever, “Language models are unsupervised multitask learners,” 2019. [En línea]. Disponible en: <https://api.semanticscholar.org/CorpusID:160025533>

- [10] T. Chen, S. Saxena, L. Li, D. J. Fleet, y G. Hinton, “Pix2seq: A language modeling framework for object detection,” 2022. [En línea]. Disponible en: <https://arxiv.org/abs/2109.10852>
- [11] P. Sharma, N. Ding, S. Goodman, y R. Soricut, “Conceptual captions: A cleaned, hypernymed, image alt-text dataset for automatic image captioning,” en *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, I. Gurevych y Y. Miyao, Eds. Melbourne, Australia: Association for Computational Linguistics, jul. 2018, pp. 2556–2565. [En línea]. Disponible en: <https://aclanthology.org/P18-1238>
- [12] L. Yu, P. Poirson, S. Yang, A. C. Berg, y T. L. Berg, “Modeling context in referring expressions,” 2016. [En línea]. Disponible en: <https://arxiv.org/abs/1608.00272>

A Apéndice

A.1 Importar librerías

Listado A.1: Librería Fine-tuning

```
1
2 import io
3 import os
4 import re
5 import json
6 import torch
7 import html
8 import base64
9 import itertools
10
11 import numpy as np
12 import supervision as sv
13
14 from IPython.core.display import display, HTML
15 from torch.utils.data import Dataset, DataLoader
16 from transformers import (
17     AdamW,
18     AutoModelForCausalLM,
19     AutoProcessor,
20     get_scheduler
21 )
22 from tqdm import tqdm
23 from typing import List, Dict, Any, Tuple, Generator
24 from peft import LoraConfig, get_peft_model
25 from PIL import Image
26 from roboflow import Roboflow
```

A.2 Cargar Modelo

Listado A.2: Código para cargar el modelo

```

1
2 CHECKPOINT = "/data/gabriel_hermosilla/patricio/florencia2/pretrained/mineria"
3 REVISION = 'refs/pr/6'
4 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
5
6 model = AutoModelForCausalLM.from_pretrained(CHECKPOINT, trust_remote_code=True,
7         revision=REVISION).to(DEVICE)
8 processor = AutoProcessor.from_pretrained(CHECKPOINT, trust_remote_code=True,
9         revision=REVISION)
10
11 print(model)

```

A.3 Cargar dirección de Conjunto de Datos

Listado A.3: Código para cargar la dirección del Dataset

```

1
2 dataset = "/data/gabriel_hermosilla/patricio/florencia2/dataset"
3
4 class JSONLDataset:
5     def __init__(self, jsonl_file_path: str, image_directory_path: str):
6         self.jsonl_file_path = jsonl_file_path
7         self.image_directory_path = image_directory_path
8         self.entries = self._load_entries()
9
10    def _load_entries(self) -> List[Dict[str, Any]]:
11        entries = []
12        with open(self.jsonl_file_path, 'r') as file:
13            for line in file:
14                data = json.loads(line)
15                entries.append(data)
16        return entries
17
18    def __len__(self) -> int:
19        return len(self.entries)
20
21    def __getitem__(self, idx: int) -> Tuple[Image.Image, Dict[str, Any]]:
22        if idx < 0 or idx >= len(self.entries):
23            raise IndexError("Index out of range")
24
25        entry = self.entries[idx]
26        image_path = os.path.join(self.image_directory_path, entry['image'])
27        try:
28            image = Image.open(image_path)
29            return (image, entry)
30        except FileNotFoundError:
31            raise FileNotFoundError(f"Image file {image_path} not found.")
32
33
34 class DetectionDataset(Dataset):
35     def __init__(self, jsonl_file_path: str, image_directory_path: str):
36         self.dataset = JSONLDataset(jsonl_file_path, image_directory_path)
37

```

```

38     def __len__(self):
39         return len(self.dataset)
40
41     def __getitem__(self, idx):
42         image, data = self.dataset[idx]
43         prefix = data['prefix']
44         suffix = data['suffix']
45         return prefix, suffix, image
46
47
48 BATCH_SIZE = 6
49 NUM_WORKERS = 0
50
51 def collate_fn(batch):
52     questions, answers, images = zip(*batch)
53     inputs = processor(text=list(questions), images=list(images), return_tensors=
54         "pt", padding=True).to(DEVICE)
55     return inputs, answers
56
57 train_dataset = DetectionDataset(
58     jsonl_file_path = f"{dataset}/Train/result.jsonl",
59     image_directory_path = f"{dataset}/Train/"
60 )
61 val_dataset = DetectionDataset(
62     jsonl_file_path = f"{dataset}/Val/result.jsonl",
63     image_directory_path = f"{dataset}/Val/"
64 )
65 train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, collate_fn=
66     collate_fn, num_workers=NUM_WORKERS, shuffle=True)
67 val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, collate_fn=collate_fn
68     , num_workers=NUM_WORKERS)

```

A.4 Configurar LoRa del Modelo

Listado A.4: Configurar el LoRa

```

1
2 config = LoraConfig(
3     r=1024,
4     lora_alpha=1024,
5     target_modules=["q_proj", "o_proj", "k_proj", "v_proj", "linear", "Conv2d", "
6         lm_head", "fc2"],
7     task_type="CAUSAL_LM",
8     lora_dropout=0.05,
9     bias="none",
10    inference_mode=False,
11    use_rslora=True,
12    init_lora_weights="gaussian",
13    revision=REVISION
14 )
15 peft_model = get_peft_model(model, config)
16 peft_model.print_trainable_parameters()

```


A.5 Realizar inferencias entre épocas

Listado A.5: Código para ver inferencias entre épocas

```

1
2 def render_inline(image: Image.Image, resize=(128, 128)):
3     """Convert image into inline html."""
4     image.resize(resize)
5     with io.BytesIO() as buffer:
6         image.save(buffer, format='jpeg')
7         image_b64 = str(base64.b64encode(buffer.getvalue()), "utf-8")
8         return f"data:image/jpeg;base64,{image_b64}"
9
10
11 def render_example(image: Image.Image, response):
12     try:
13         detections = sv.Detections.from_lmm(sv.LMM.FLORENCE_2, response,
14             resolution_wh=image.size)
15         image = sv.BoxAnnotator(color_lookup=sv.ColorLookup.INDEX).annotate(image
16             .copy(), detections)
17         image = sv.LabelAnnotator(color_lookup=sv.ColorLookup.INDEX).annotate(
18             image, detections)
19     except:
20         print('failed to redner model response')
21     return f"""
22 <div style="display: inline-flex; align-items: center; justify-content: center;">
23 
25 <p style="width:512px; margin:10px; font-size:small;">{html.escape(json.dumps
26     (response))}</p>
27 </div>
28 """
29
30 def render_inference_results(model, dataset: DetectionDataset, count: int):
31     html_out = ""
32     count = min(count, len(dataset))
33     for i in range(count):
34         image, data = dataset.dataset[i]
35         prefix = data['prefix']
36         suffix = data['suffix']
37         inputs = processor(text=prefix, images=image, return_tensors="pt").to(
38             DEVICE)
39         generated_ids = model.generate(
40             input_ids=inputs["input_ids"],
41             pixel_values=inputs["pixel_values"],
42             max_new_tokens=1024,
43             num_beams=3
44         )
45         generated_text = processor.batch_decode(generated_ids,
46             skip_special_tokens=False)[0]
47         answer = processor.post_process_generation(generated_text, task='<OD>',
48             image_size=image.size)
49         html_out += render_example(image, answer)
50
51     display(HTML(html_out))

```

A.6 Definir Loop de Entrenamiento Fine-tuning

Listado A.6: Código del loop de entrenamiento

```

1
2 def train_model(train_loader, val_loader, model, processor, epochs=10, lr=1e-6):
3     optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
4     num_training_steps = epochs * len(train_loader)
5     lr_scheduler = get_scheduler(
6         name="linear",
7         optimizer=optimizer,
8         num_warmup_steps=0,
9         num_training_steps=num_training_steps,
10    )
11
12    render_inference_results(peft_model, val_loader.dataset, 6)
13
14    for epoch in range(epochs):
15        model.train()
16        train_loss = 0
17        for inputs, answers in tqdm(train_loader, desc=f"Training Epoch {epoch +
18            1}/{epochs}"):
19
20            input_ids = inputs["input_ids"]
21            pixel_values = inputs["pixel_values"]
22            labels = processor.tokenizer(
23                text=answers,
24                return_tensors="pt",
25                padding=True,
26                return_token_type_ids=False
27            ).input_ids.to(DEVICE)
28
29            outputs = model(input_ids=input_ids, pixel_values=pixel_values,
30                labels=labels)
31            loss = outputs.loss
32
33            loss.backward(), optimizer.step(), lr_scheduler.step(), optimizer.
34                zero_grad()
35            train_loss += loss.item()
36
37            #Muestra la p rdida del entrenamiento en cada poca
38            avg_train_loss = train_loss / len(train_loader)
39            print(f"Average Training Loss: {avg_train_loss}")
40
41            model.eval()
42            val_loss = 0
43            with torch.no_grad():
44                for inputs, answers in tqdm(val_loader, desc=f"Validation Epoch {
45                    epoch + 1}/{epochs}"):
46
47                    input_ids = inputs["input_ids"]
48                    pixel_values = inputs["pixel_values"]
49                    labels = processor.tokenizer(
50                        text=answers,
51                        return_tensors="pt",
52                        padding=True,
53                        return_token_type_ids=False
54                    ).input_ids.to(DEVICE)
55
56                    outputs = model(input_ids=input_ids, pixel_values=pixel_values,

```

```

53         labels=labels)
54         loss = outputs.loss
55
56         val_loss += loss.item()
57
58         avg_val_loss = val_loss / len(val_loader)
59         #Muestra la media de p rdida del set de validaci n
60         print(f"Average Validation Loss: {avg_val_loss}")
61
62         render_inference_results(peft_model, val_loader.dataset, 6)
63
64         #Se crea un checkpoint en cada una de las pocas
65         output_dir = f"./model_checkpoints/epoch_{epoch+1}"
66         os.makedirs(output_dir, exist_ok=True)
67         model.save_pretrained(output_dir)
68         processor.save_pretrained(output_dir)

```

A.7 Ejecutar entrenamiento

Listado A.7: Código para iniciar el entrenamiento

```

1
2 EPOCHS = 10
3 LR = 5e-6
4
5 train_model(train_loader, val_loader, peft_model, processor, epochs=EPOCHS, lr=LR
6 )

```

A.8 Recolectar predicciones para métricas

Listado A.8: Código para recolectar predicciones del modelo fine-tuned

```

1
2 CLASES = ["bem", "ripio", "tranque"]
3
4 def evaluate_model(model, val_dataset, processor, classes, device):
5     targets = []
6     predictions = []
7
8     for i in tqdm(range(len(val_dataset.dataset)), desc="Evaluating"):
9         image, data = val_dataset.dataset[i]
10        prefix = data['prefix']
11        suffix = data['suffix']
12
13        inputs = processor(text=prefix, images=image, return_tensors="pt").to(
14            device)
15        generated_ids = model.generate(
16            input_ids=inputs["input_ids"],
17            pixel_values=inputs["pixel_values"],
18            max_new_tokens=1024,
19            num_beams=3
20        )
21        generated_text = processor.batch_decode(generated_ids,
22            skip_special_tokens=False)[0]

```

```

22     prediction = processor.post_process_generation(generated_text, task='<OD>
    ', image_size=image.size)
23     prediction = sv.Detections.from_lmm(sv.LMM.FLORENCE_2, prediction,
    resolution_wh=image.size)
24     prediction = prediction[np.isin(prediction['class_name'], classes)]
25     prediction.class_id = np.array([classes.index(class_name) for class_name
    in prediction['class_name']])
26     prediction.confidence = np.ones(len(prediction))
27
28     target = processor.post_process_generation(suffix, task='<OD>',
    image_size=image.size)
29     target = sv.Detections.from_lmm(sv.LMM.FLORENCE_2, target, resolution_wh=
    image.size)
30     target.class_id = np.array([classes.index(class_name) for class_name in
    target['class_name']])
31
32     targets.append(target)
33     predictions.append(prediction)

```

A.9 Calcular Mean Average Precision

Listado A.9: Código para calcular el MaP

```

1
2     mean_average_precision = sv.MeanAveragePrecision.from_detections(
3         predictions=predictions,
4         targets=targets,
5     )
6
7     print(f"map50_95: {mean_average_precision.map50_95:.2f}")
8     print(f"map50: {mean_average_precision.map50:.2f}")
9     print(f"map75: {mean_average_precision.map75:.2f}")

```

A.10 Guardar Modelo Fine-tuned

Listado A.10: Código para Guardar el Modelo Fine-tuned

```

1
2     output_dir = "modelo_nuevo"
3
4     peft_model.save_pretrained(output_dir)
5     processor.save_pretrained(output_dir)

```